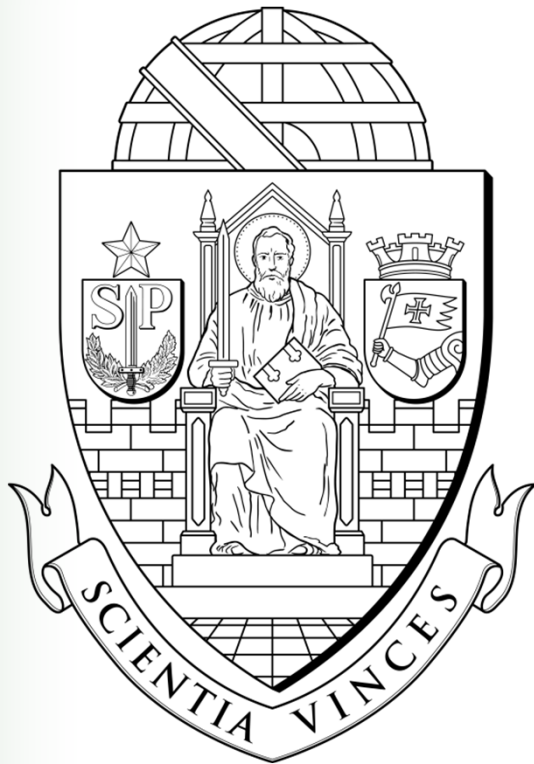


Árvores B

Variações



- Nesta apresentação serão vistas algumas variações de implementação de árvores B

Variações de Árvores B

□ Árvores B virtuais

- Parte da árvore é mantida em memória principal

□ Árvores B*

- Há uma exigência que $2/3$ de cada nó esteja preenchido, diferente de uma árvore B que exige apenas $1/2$ de cada nó ocupado

□ Árvores B+

- Mantém todas as chaves (registros) nas folhas

Árvores B Virtuais

- ❑ Apesar de Árvores B serem muito eficientes, seu desempenho pode ainda ser melhorado
- ❑ O fato de não ser possível manter todo o índice na memória principal não significa que não se possa manter pelo menos parte dele

Árvores B Virtuais

- ❑ Assuma um índice que:
 - ocupa 1 Mb
 - existem disponíveis apenas 256 Kb de RAM
 - um nó (página) ocupa 4 Kb com $m=64$ chaves por nó
- ❑ Este índice pode ser armazenado em uma árvore B totalmente contida em 3 níveis
- ❑ Dessa forma, é possível chegar a qualquer nó com, no máximo, 3 acessos a disco
- ❑ Entretanto, se a raiz for mantida todo o tempo em memória principal, ainda sobraria muito espaço em RAM
- ❑ Com essa solução, número de acessos no pior caso diminui em uma unidade

Árvores B Virtuais

- ❑ É possível generalizar esta ideia e ocupar toda a memória disponível com quantos nós forem possíveis
- ❑ Assim, quando um nó for necessário ele pode já estar em memória principal
- ❑ Se não estiver, ele é carregado para a memória, substituindo uma página que estava em memória
- ❑ Tem-se um RAM *buffer* também conhecido como **árvore B virtual**

Árvores B Virtuais

- ❑ Política de gerenciamento de substituição de nós (LRU)
 - Se o nó não estiver em RAM, e esta estiver cheia, um nó em RAM precisa ser substituído
 - LRU (*Last Recently Used*)
 - ❖ substitui-se o nó que foi acessado menos recentemente
- ❑ O processo de acessar o disco para trazer um nó que não está no RAM *buffer* é denominado *page fault*

Árvores B Virtuais

- ❑ Uma opção seria colocar todos os níveis mais altos da árvore em RAM
- ❑ No exemplo de 256 Kb de RAM e nós de 4Kb, podemos manter 64 nós (páginas) em memória
- ❑ Isso comporta a raiz e cerca de 8-10 nós do segundo nível, restando ainda espaço (utiliza-se LRU) sendo que o número de acessos diminui em mais uma unidade
- ❑ Na verdade, a criação de *buffers* é incluída em qualquer aplicação real de utilização de árvores B

Árvores B Virtuais

- ❑ Com relação à informação associada às chaves, ou seja, os demais campos dos registros:
 - Se a informação for mantida junto com a chave, ganha-se um acesso a disco, mas perde-se no número de chaves que pode ser colocado em uma página; isso reduz a ordem da árvore, e aumenta a sua altura
 - Se ficar em um arquivo separado, a árvore é realmente usada como índice, e cada chave tem o ponteiro da posição do registro associado no arquivo principal de dados

Árvores B*

- Uma árvore T do tipo B* e ordem m é uma árvore de busca em m -vias, que é ou vazia ou de altura $h \geq 1$ e satisfaz as seguintes propriedades:
 - (a) a raiz tem pelo menos 2 filhos e no máximo $2^{\lfloor (2m-2)/3 \rfloor} + 1$
 - (b) todos os nós, exceto a raiz e nós de falha, têm pelo menos $2^{\lceil (2m-2)/3 \rceil}$ e no máximo m filhos
 - (c) todos os nós de falha estão no mesmo nível
- $\lfloor x \rfloor$ é o piso de x , ou seja, é o maior inteiro $\leq x$
- $\lceil x \rceil$ é o teto de x , ou seja, é o menor inteiro $\geq x$

Árvores B+

- ❑ Outra variação para a implementação de árvores B é conhecida como B+
- ❑ Em uma árvore B+ todas as chaves (registros) são armazenadas no último nível (nas folhas)
- ❑ Os níveis acima do último nível constituem um índice organizado como árvore B

Acesso Sequencial Indexado (ISAM)

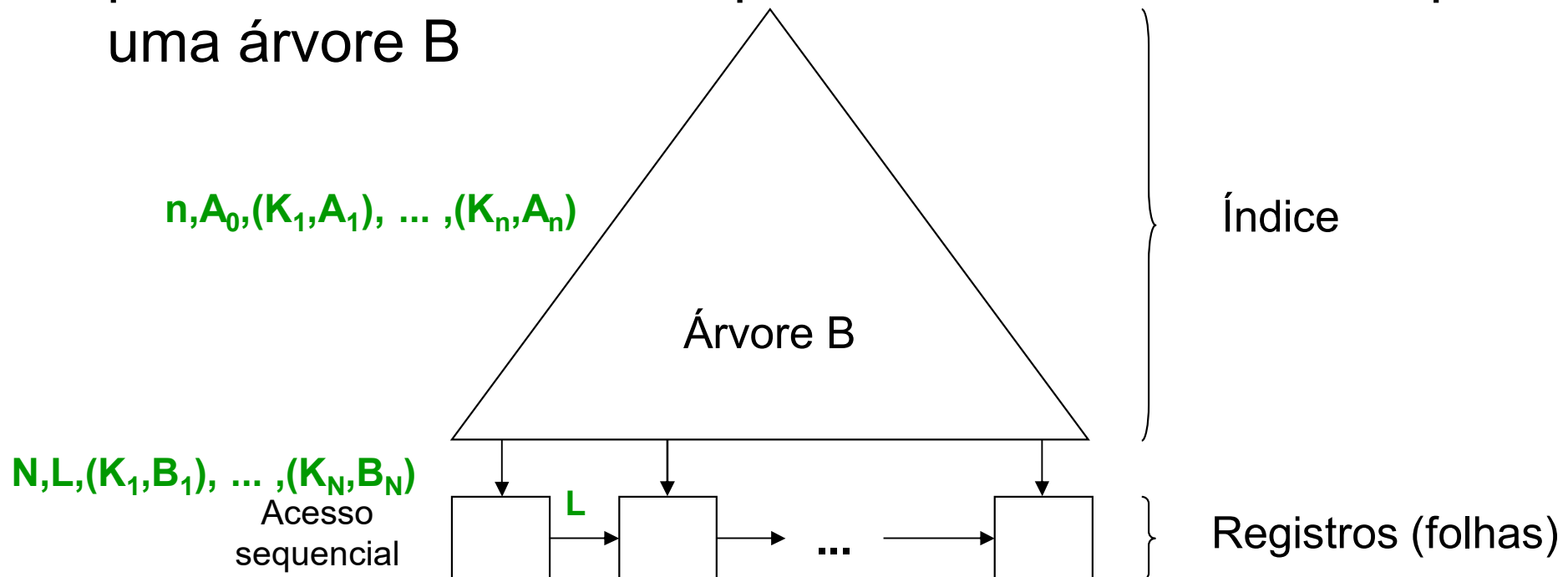
- ❑ Em uma árvore B não é possível fazer acesso sequencial eficiente
- ❑ Como arquivos devem suportar tanto acesso indexado eficiente como acesso sequencial o conceito de árvores B foi estendido para árvores B+
- ❑ ISAM: *Indexed Sequential Access Method*
- ❑ **Sequence set**: um conjunto de registros ordenados (segundo alguma chave), que podem ser recuperados com um acesso
 - Inviável manter todo o arquivo ordenado, mas é possível manter blocos

Acesso Sequencial Indexado (ISAM)

- ❑ Como visto, um bloco é a menor unidade básica de I/O
 - O tamanho do bloco é determinado pelo tamanho dos *buffers* de I/O
- ❑ Assim, um arquivo pode ser mantido como uma sequência lógica de blocos, ou seja, um *sequence set*
 - O conteúdo de um bloco está ordenado e pode ser recuperado em um único acesso
 - Cada bloco mantém um ponteiro para a posição do próximo bloco (na sequência) no disco
 - Blocos logicamente adjacentes não estão (necessariamente) fisicamente adjacentes
 - Garantia de acesso sequencial ao arquivo

Árvores B+

- ❑ No índice aparecem somente as chaves
- ❑ Nas folhas estão todas as chaves (ou todos os registros) do arquivo
- ❑ As folhas são conectadas da esquerda para a direita, permitindo o acesso sequencial mais eficiente do que em uma árvore B



Árvore B x Árvore B+

□ Árvore B:

- $n, A_0, (K_1, A_1, B_1), (K_2, A_2, B_2), \dots, (K_n, A_n, B_n)$

□ Árvore B+

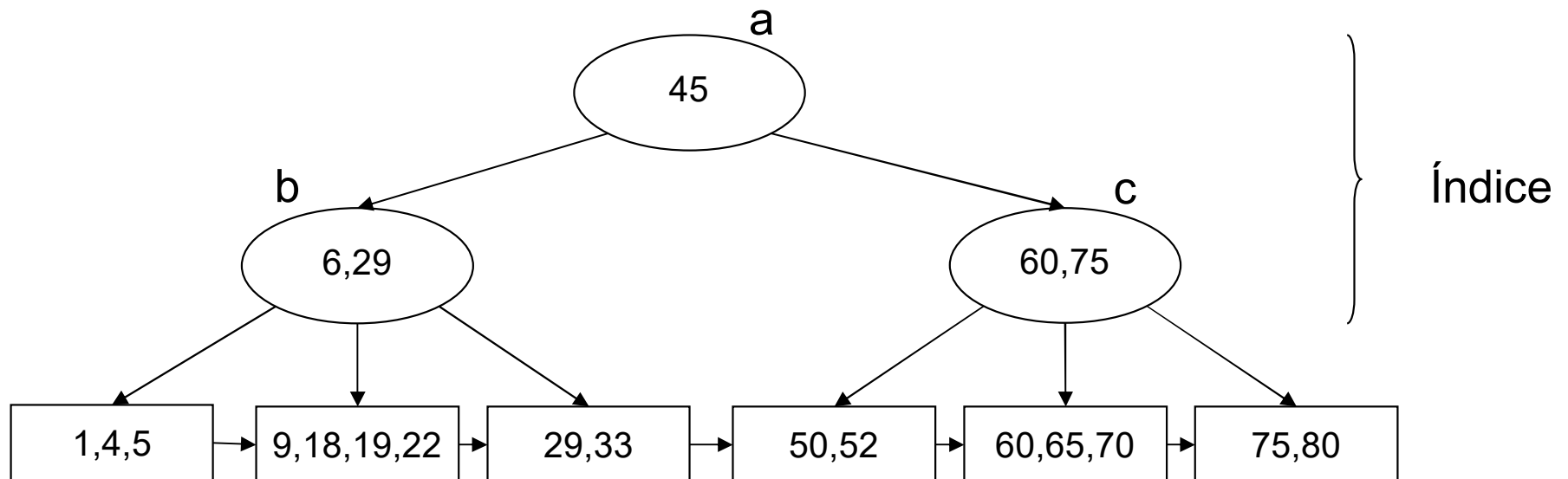
- Índice (Árvore B)
 - ❖ $n, A_0, (K_1, A_1), \dots, (K_n, A_n)$
- Registros (folhas)
 - ❖ $N, L, (K_1, B_1), \dots, (K_N, B_N)$

□ Onde:

- n : número de chaves no respectivo nó do índice
- N : número de chaves na respectiva folha (bloco)
- K_i : Chaves do índice ou da folha
- A_i : Endereços dos registros no índice (árvore B)
- B_i : Endereços dos registros no arquivo principal
- L : endereço do próximo bloco em disco

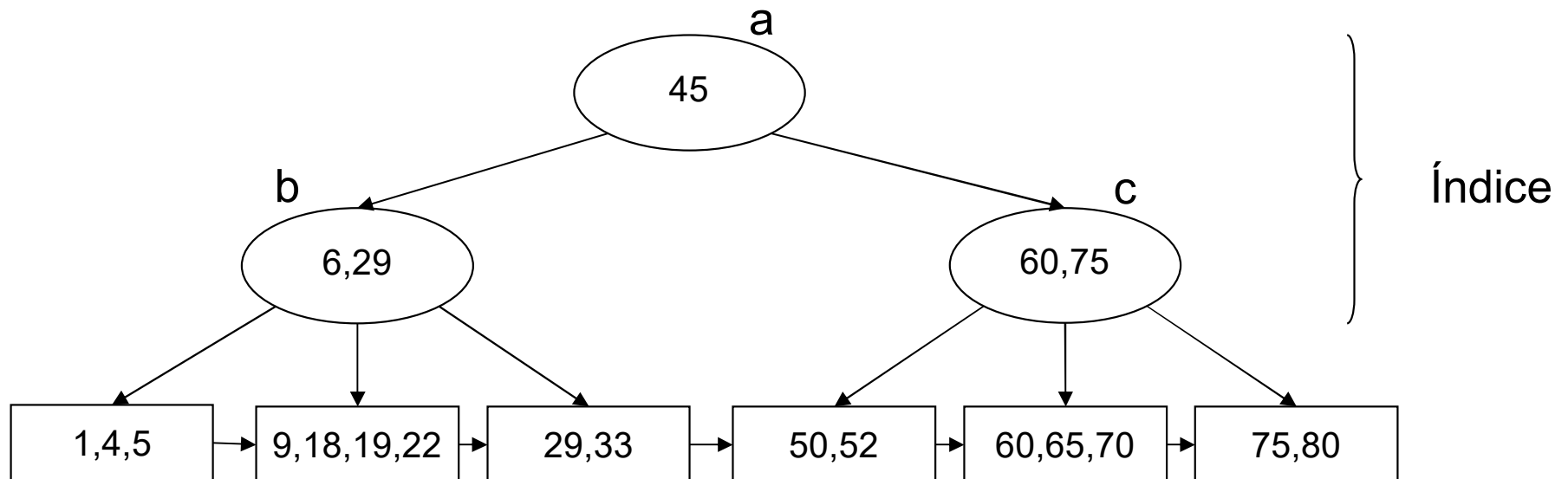
Árvores B+

- ❑ Na busca por uma chave X, o processo tem início pela raiz e continua até uma folha
- ❑ Como todos os registros residem nas folhas, a pesquisa não pára se X for encontrado em um nó do índice
 - Neste caso, o filho à direita é seguido até que uma folha seja encontrada



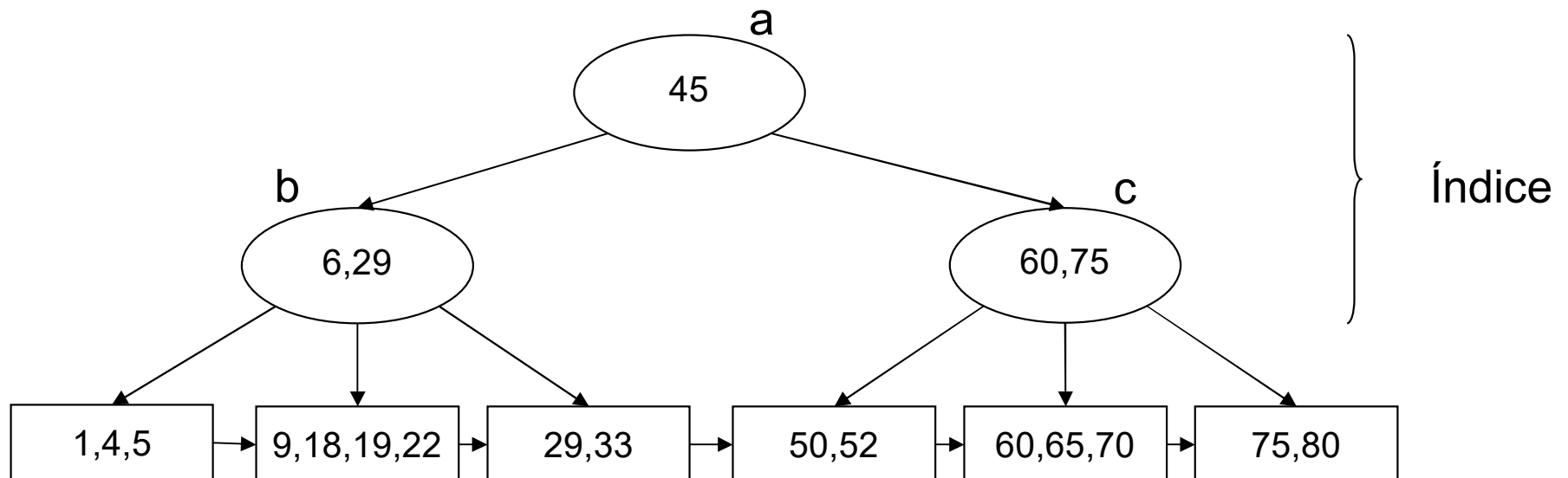
Árvores B+

- ❑ Por exemplo, na busca das chaves 29, 60 e 75 (que aparecem tanto no índice como nas folhas) os valores encontrados no índice são usados apenas para conduzir até a folha correta



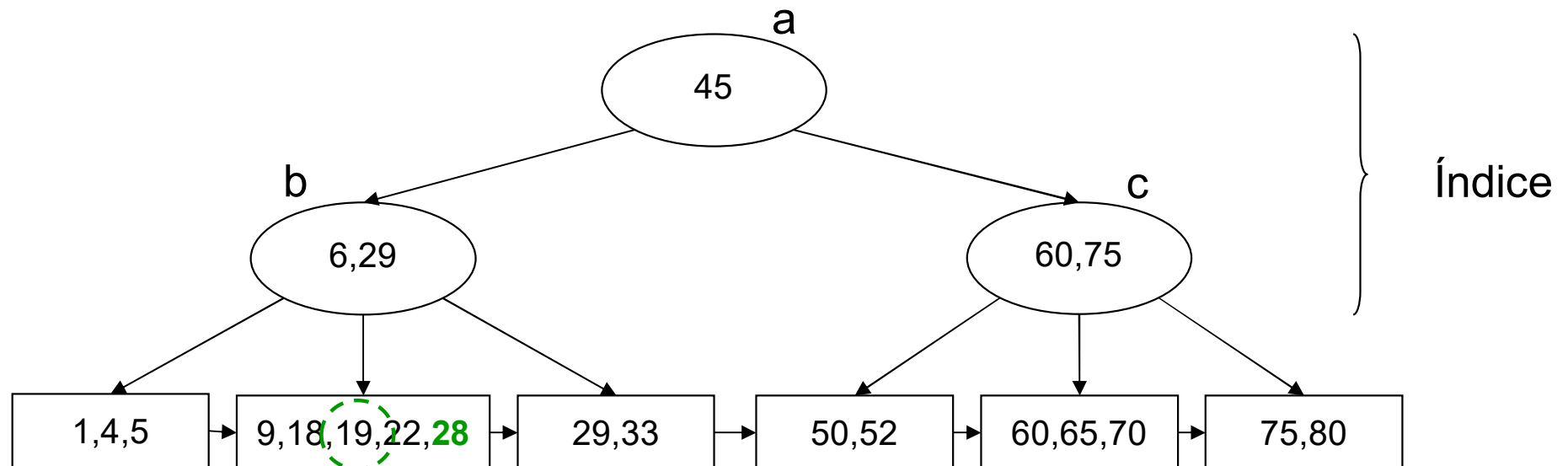
Árvores B+

- ❑ A inserção em uma árvore B+ é similar àquela em uma árvore B
- ❑ A diferença reside em, quando uma folha é dividida em duas, o algoritmo efetua uma cópia da chave que pertence ao registro do meio para o nó pai no nível anterior, ficando o registro do meio na folha da direita



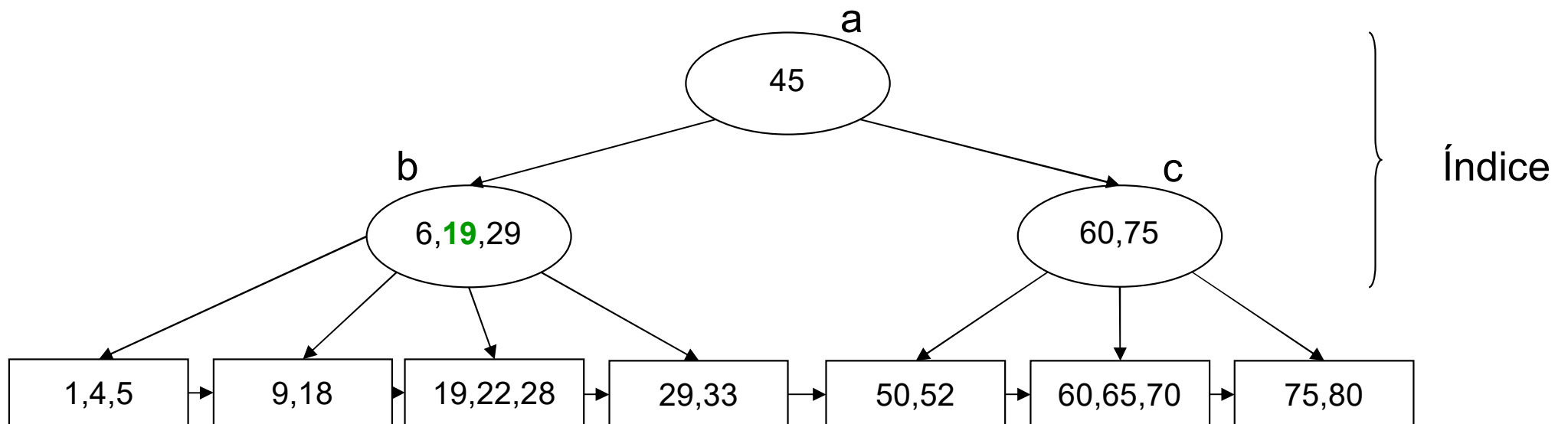
Árvores B+

- ❑ No exemplo, ao inserir o registro 28 a folha fica grande demais
- ❑ Assim, a folha é dividida e registro 19 vai para a folha da direita
- ❑ A chave 19 é copiada para o nó pai



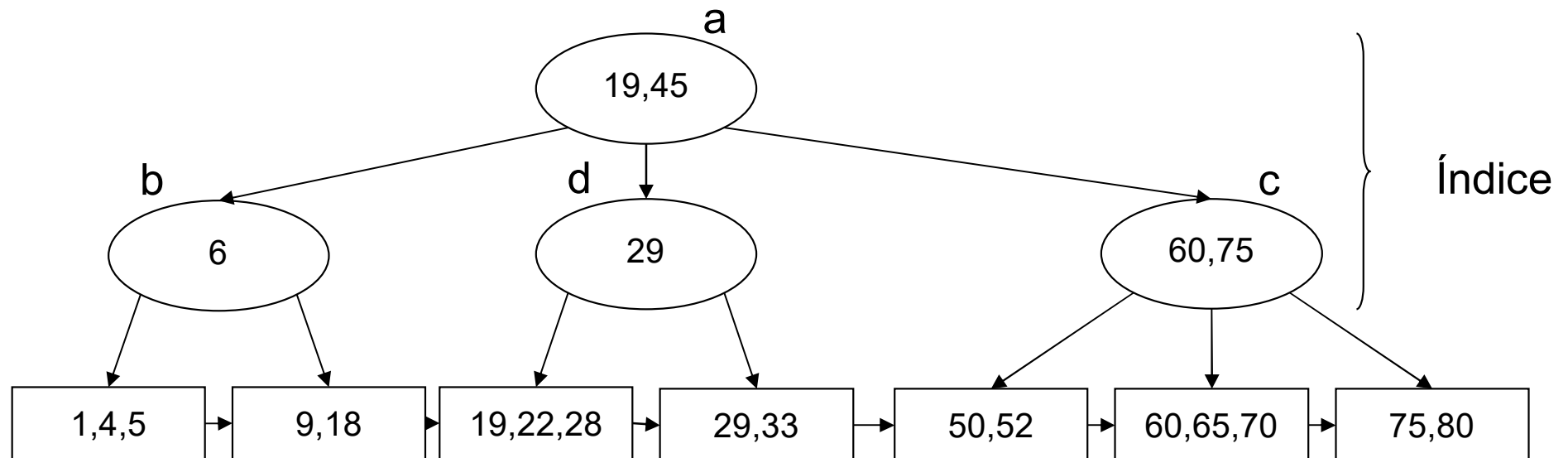
Árvores B+

- ❑ O demais passos são similares à inserção em árvore B



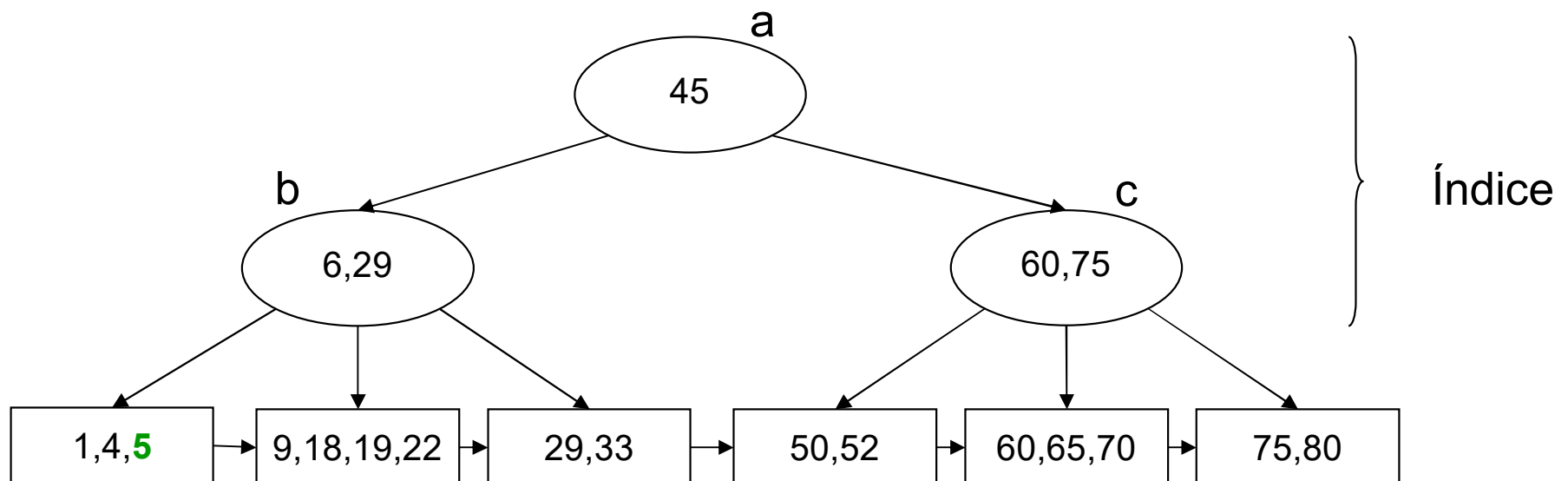
Árvores B+

- ❑ O demais passos são similares à inserção em árvore B



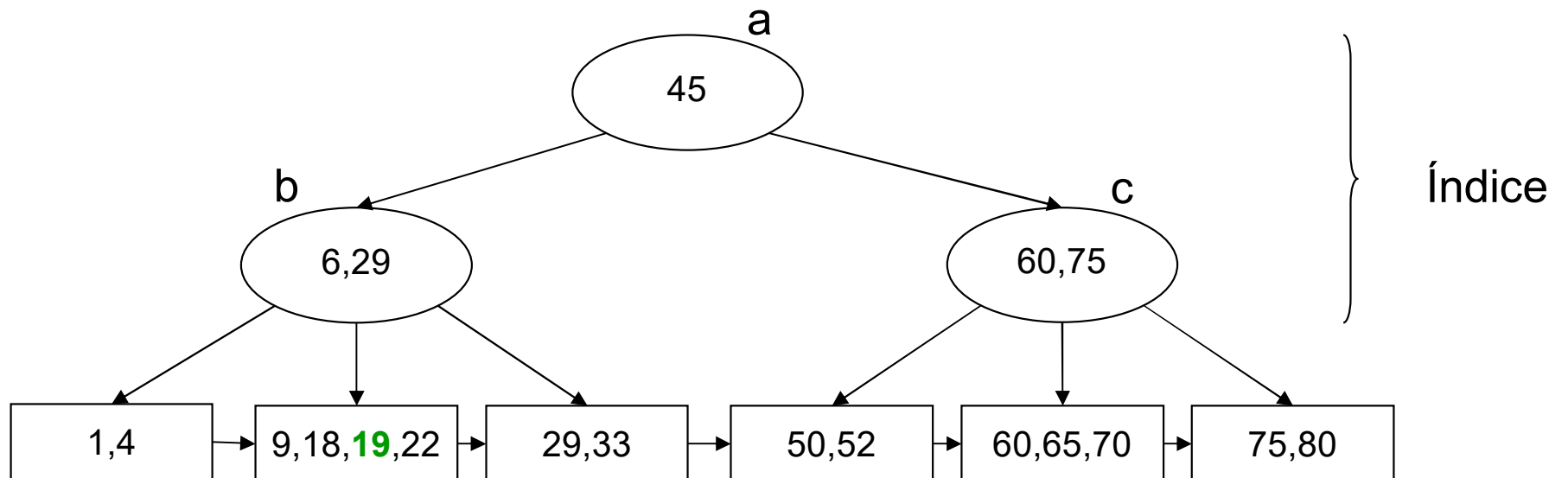
Árvores B+

- ❑ A remoção é mais simples do que em uma árvore B
- ❑ O registro a ser retirado encontra-se sempre em uma folha, não havendo necessidade procurar um elemento para substituí-lo
- ❑ Desde que a folha fique pelo menos com a metade dos registros necessários, o índice não precisa ser modificado, mesmo se uma cópia da chave pertencendo ao registro sendo removido esteja no índice



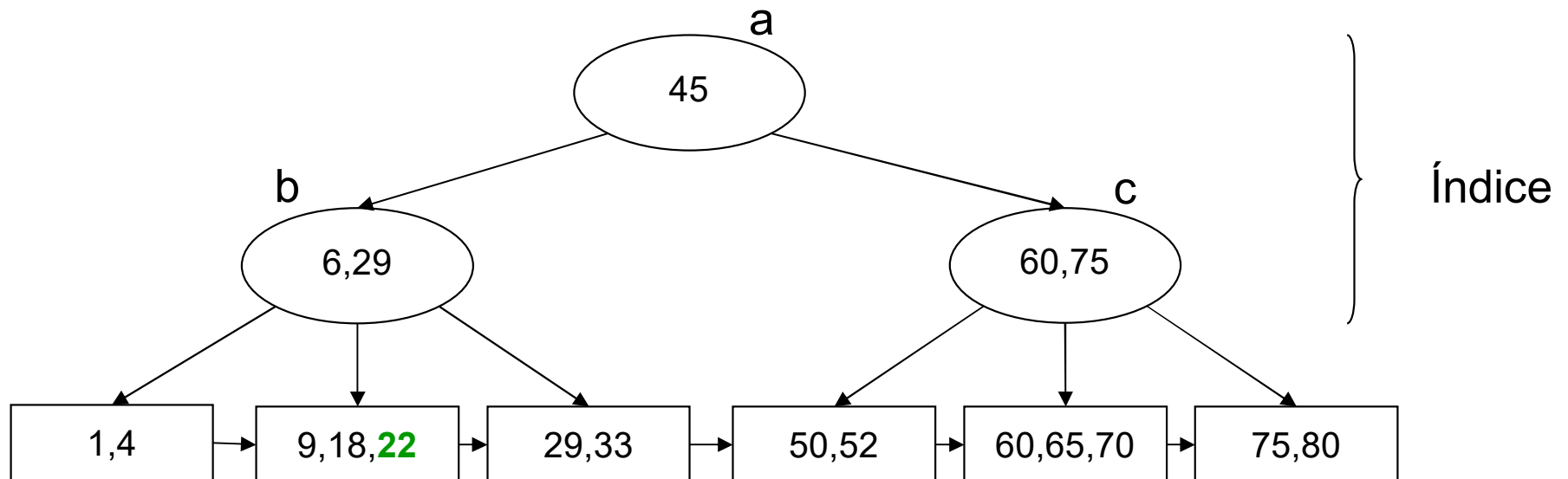
Árvores B+

- ❑ No exemplo, removendo-se os registros 5



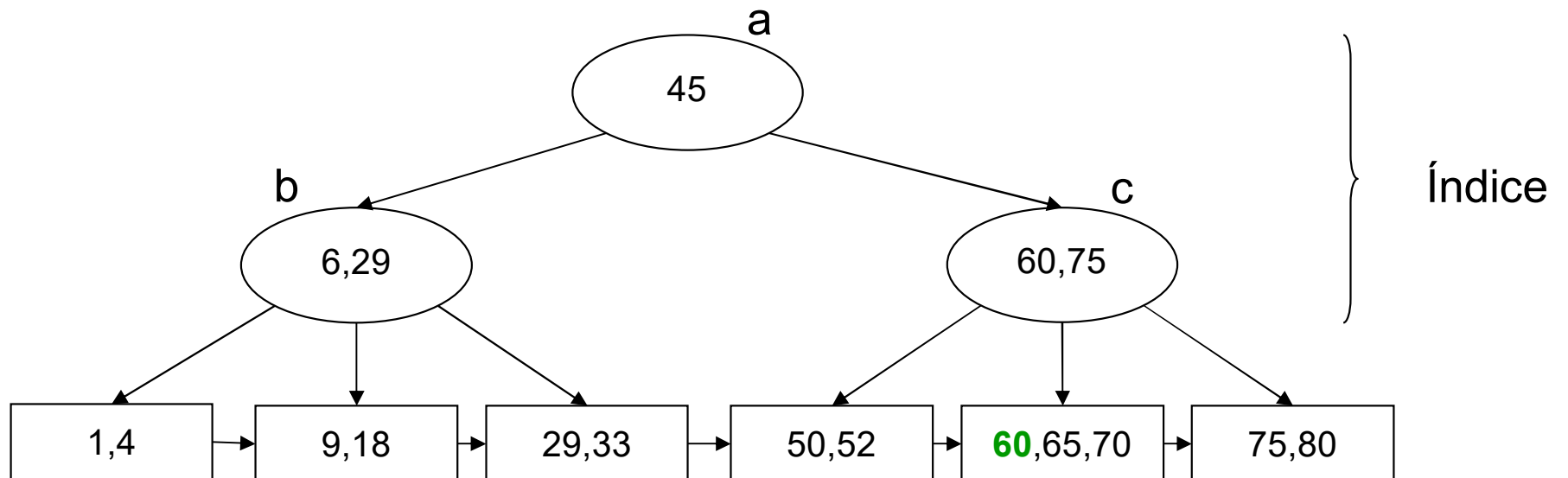
Árvores B+

❑ No exemplo, removendo-se os registros 5, 19



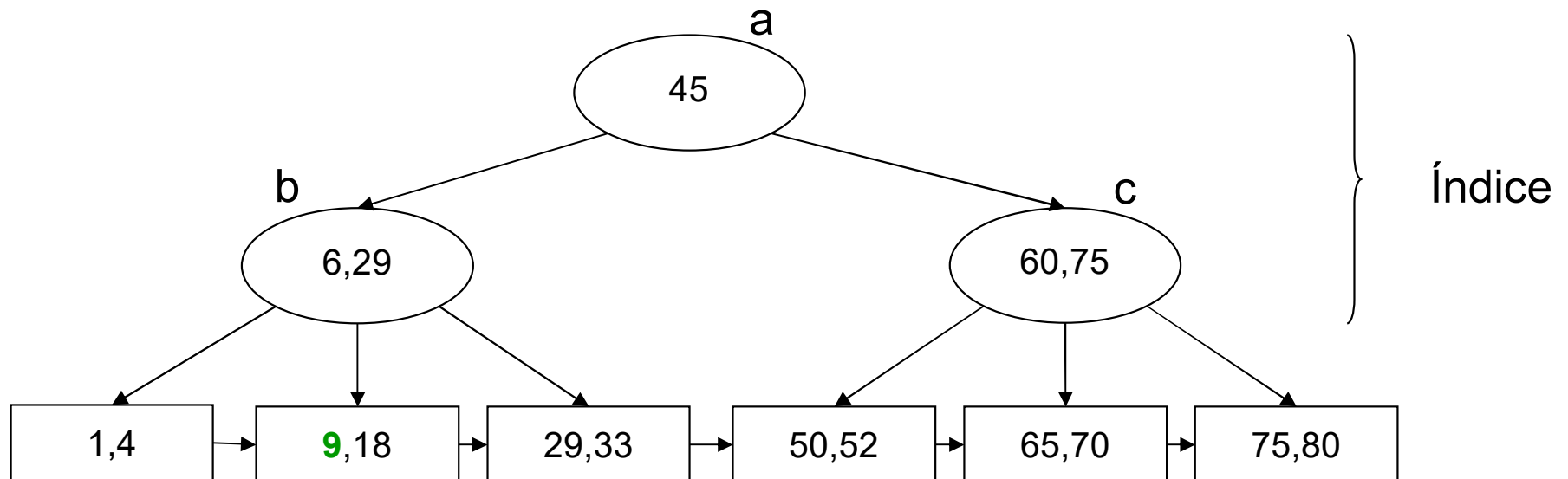
Árvores B+

- ❑ No exemplo, removendo-se os registros 5, 19, 22



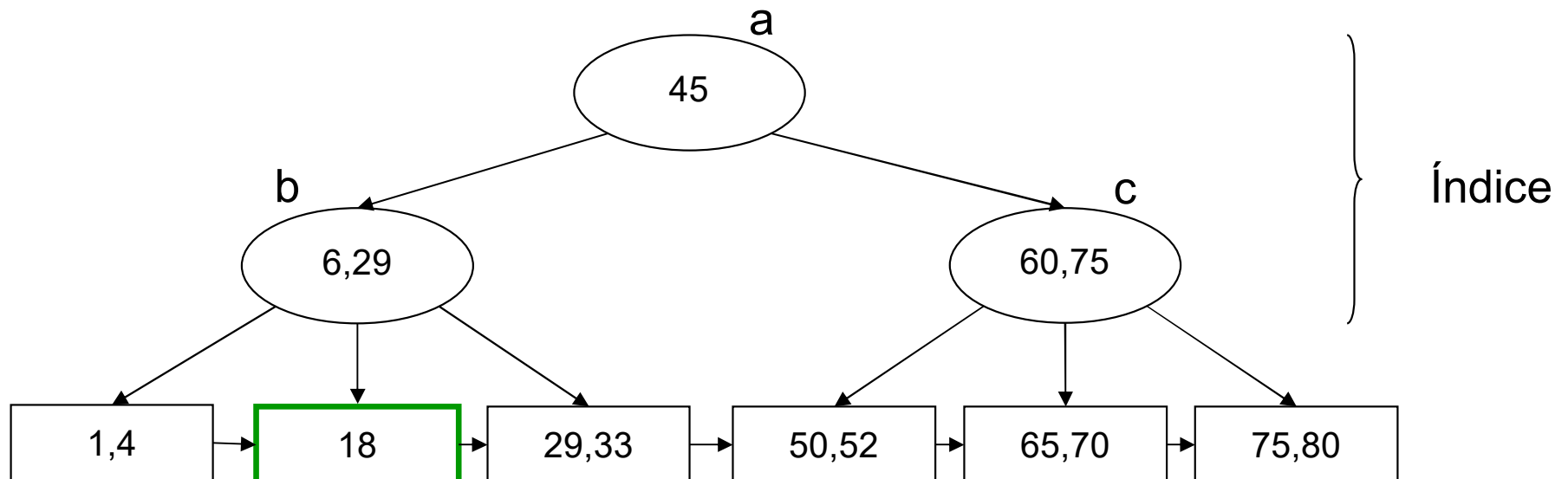
Árvores B+

- ❑ No exemplo, removendo-se os registros 5, 19, 22 e 60 não há alteração no índice
- ❑ Removendo o registro 9...



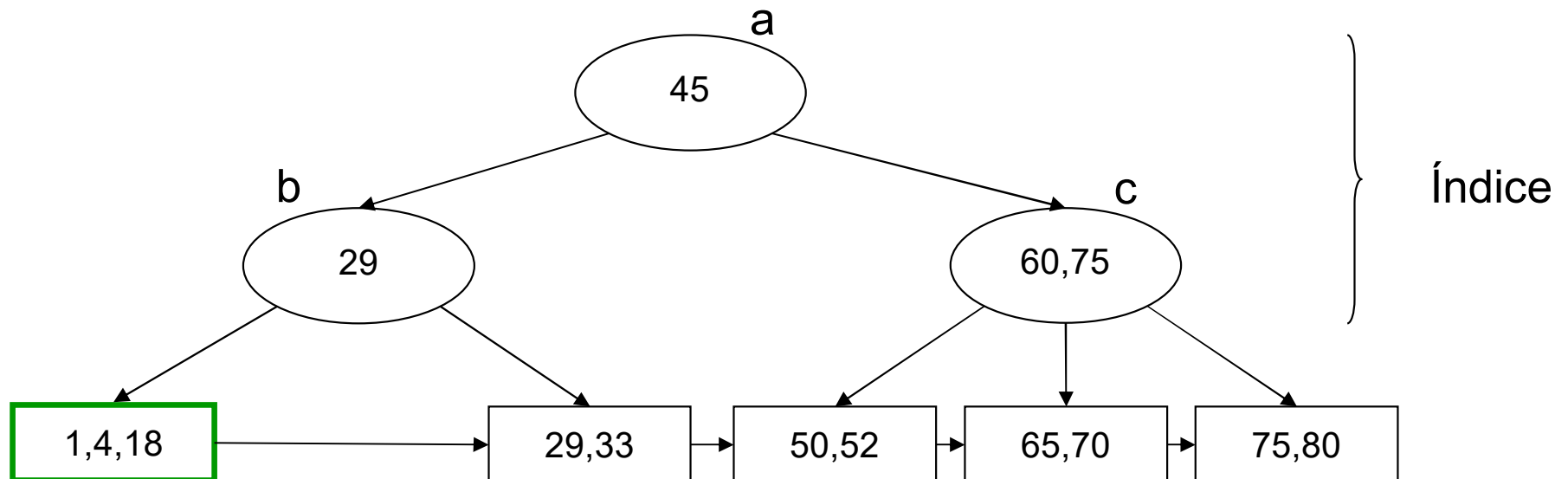
Árvores B+

- ❑ No exemplo, removendo-se os registros 5, 19, 22 não há alteração no índice
- ❑ Removendo o registro 9 a folha fica com poucos registros e precisa ser agrupada com outra folha



Árvores B+

- ❑ No exemplo, removendo-se os registros 5, 19, 22 não há alteração no índice
- ❑ Removendo o registro 9 a folha fica com poucos registros e precisa ser agrupada com outra folha



Resumo

□ Árvore B Virtual

- Parte da estrutura permanece na memória principal para acesso mais rápido

□ Árvore B*

- Maior ocupação dos nós internos: exige no mínimo $2/3$ de preenchimento, melhorando aproveitamento de espaço

□ Árvore B+

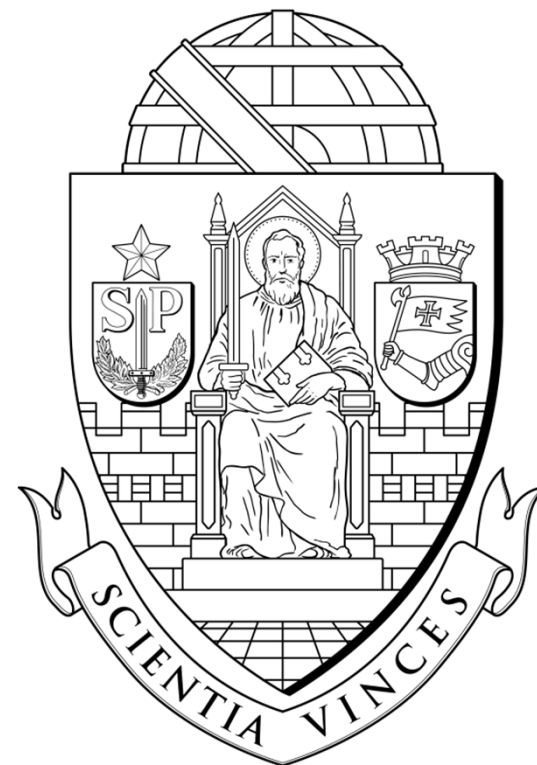
- Todas as chaves/valores ficam nas folhas, nós internos armazenam apenas referências para navegação

Copyright© Apresentação 2025 por
José Augusto Baranauskas
Universidade de São Paulo

Professores são convidados a utilizarem esta apresentação da maneira que lhes for conveniente para uso não comercial, desde que esta nota de *copyright* permaneça intacta.

Slides baseados em:

- ❑ Horowitz, E. & Sahni, S.; Fundamentos de Estruturas de Dados, Editora Campus, 1984.
- ❑ Wirth, N.; Algoritmos e Estruturas de Dados, Prentice/Hall do Brasil, 1989.



USP